FINDTHATSHIT

Product Brief & Scope of Work (SOW)

AI-Powered Social Resource Library

FindThatShit is an AI-powered resource management platform that helps users save, organize, understand, and retrieve

valuable resources discovered through Instagram, X (Twitter), and other online platforms.

The core problem is simple: users discover useful templates, PDFs, tools, Notion pages, Drive folders, websites and other resources through social media, but later struggle to remember where they found them or how to locate them. FindThatShit stores the resource together with its source and AI-generated context, making it searchable later.

Product principle: FindThatShit is not just a bookmarking tool. It is a personal memory layer for valuable resources discovered online.

1. MVP OBJECTIVE

The MVP is a web application focused on one core workflow:

Save a resource → Understand it → Organize it → Find it later.

Users independently complete any required Instagram/X follow, comment, like, or DM actions. After receiving the resource, they add its source and resource to FindThatShit.

2. PRIMARY USER FLOW

1. Discover

User finds a useful Reel/Post and completes any required follow/comment/DM action on the social platform.

2. Add Entry

User opens FindThatShit and selects Add Entry.

3. Source

Choose Reel, Post, X/Twitter Post, or Other; add source URL and optionally creator name.

4. Resource

Paste the resource URL or upload a PDF/DOC/document.

5. Storage

Uploaded files are stored in Amazon S3; external URLs are stored as resource references.

6. AI Processing

FindThatShit extracts available source content, transcribes Reel audio where possible, processes written posts/documents, and generates descriptions/tags.

7. Library

The processed entry appears in the user's searchable resource library.

8. Retrieval

User later searches by keyword, metadata, description, or eventually by natural-language voice query.

3. ADD ENTRY FORM

The dashboard will display a list/grid of saved resources. An Add Entry button opens a modal/page containing the following fields:

- Source type: Instagram Reel, Instagram Post, X/Twitter Post, Other.
- Source URL: Link to the original Reel/Post.
- Creator name: Optional; may be extracted where technically possible.
- Resource URL: Google Drive, Google Docs, Notion, website, GitHub, Dropbox, or another external URL.
- Upload resource: Upload PDF, DOC/DOCX, TXT, or other supported document formats.
- Optional metadata: User-entered title/tags where desired.

4. AI PROCESSING

Instagram Reel: Reel/source information → audio extraction or available source content → Speech-to-Text → transcript → LLM analysis.

Written Post: Post text → content processing → LLM analysis.

Uploaded Document: File → text/document extraction → LLM analysis.

The LLM generates a concise resource description, source/content summary, keywords, and tags. This metadata is stored with the resource and later used for search and semantic retrieval.

- Resource description: What the resource is, what it contains, and what it is useful for.
- Source description: What the Reel/Post is about and how it relates to the resource.
- Transcript/content: Reel audio transcript or extracted written post content.
- Tags/topics: Examples include Resume, Python, Notion, Interview Prep, DevOps, Templates.
- Processing status: Uploaded → Processing → Processed / Failed.

5. RESOURCE LIBRARY

The main dashboard should present each saved resource as a compact card/list item.

- Resource title and AI-generated description.
- Creator and source platform.
- Source/Reel/Post link.
- Resource link or uploaded-file access.
- Tags/topics.
- Date added.
- Processing status.
- Actions such as Open Resource, View Source, and View Details.

6. RESOURCE DETAIL VIEW

- Source: platform, creator, source URL, source type.
- Source content: Reel transcript or post text.
- Resource: title, description, resource type, URL/file.
- AI metadata: tags, keywords, generated summaries.
- System metadata: date added, processing status, last processed timestamp.

7. SEARCH & RETRIEVAL

The MVP should support keyword and metadata search across resource titles, creators, tags, descriptions, transcripts, post content, and resource types.

A second layer should support semantic retrieval so that users can search by meaning rather than exact filenames. For example, a query such as “the resume template I saved for software engineering applications” should be able to retrieve a file named Modern_Resume_Final.pdf when its stored context matches.

The retrieval architecture should therefore be designed for future hybrid search using conventional database search plus embeddings/vector search.

8. VOICE SEARCH

Voice search is a major product differentiator and should be planned into the architecture from the beginning.

Future flow: Microphone → Gnani STT → Query Understanding / LLM → Semantic Retrieval → Result → TTS.

Example query: “Find the Python cheat sheet I saved from Instagram last month.”

The system converts the query to text, searches resource descriptions, source transcripts, tags and metadata, ranks relevant resources, and returns the most useful results. TTS can then provide a spoken response.

This creates the intended STT → LLM → Retrieval → TTS experience.

9. STORAGE ARCHITECTURE

Database: PostgreSQL for structured metadata, users, resource entries, source content, tags, processing status, and file references.

Object storage: Amazon S3 for uploaded PDFs/documents and other supported binary resources. The database stores the S3 object key/reference rather than binary content.

Access: Generate secure, time-limited URLs when users request stored files.

Suggested object structure: findthatshit/users/{user_id}/resources/{resource_id}/resource.ext

10. CORE DATA MODEL

Entity

Important Fields

User

id, name, email, created_at

Resource Entry

id, user_id, title, creator, platform, source_url, resource_url, resource_type, description, tags,

date_added, processing_status

Source Content

entry_id, source_type, transcript, post_text, extracted_metadata

File

entry_id, s3_object_key, filename, mime_type, file_size, created_at

11. ASYNCHRONOUS PROCESSING

AI processing should run asynchronously so large files or transcription jobs do not block the user's request.

Submission: Web App → API → Database/S3 → Processing Queue

Processing: Source/File → Content Extraction → STT if audio → LLM → Description + Tags + Metadata → Database

Recommended processing states: UPLOADED → PROCESSING → PROCESSED, with FAILED for recoverable

processing errors.

12. MVP FEATURE SCOPE

- Authentication: registration, login, logout, basic profile.
- Dashboard: resource list/grid, sorting, filtering, search.
- Add Entry: source type, source URL, creator, resource URL, file upload.
- Storage: external resource URLs and S3-backed uploaded files.
- AI processing: source content processing, Reel transcription where available, document/post text extraction, descriptions, summaries, tags and keywords.
- Resource detail: complete source/resource/AI metadata view.
- Search: keyword, creator, tag, description and source-content search.
- Semantic retrieval foundation: embeddings/vector-ready representation of descriptions, transcripts and tags.
- Voice UI foundation: microphone interaction and backend architecture prepared for Gnani STT → retrieval → TTS.

13. OUT OF SCOPE FOR MVP

- Automatic Instagram/X login or account control.
- Automatic follow, like, comment, or DM.
- Automatic Instagram inbox reading.
- Temporary/proxy social-media accounts.
- Automatic comment deletion.
- Fully automated social-media resource claiming.
- Unofficial browser automation against social platforms.
- Complex platform-specific integrations before the core resource-library workflow is validated.

14. FUTURE SCOPE

- Direct social sharing: Share a Reel/Post directly to FindThatShit.
- Automatic source understanding: creator, resource name, claim keyword, follow requirement, topic and source transcript extraction.
- Guided claim assistant: step-by-step instructions for completing follow/comment/DM requirements while keeping the user in control.
- Native mobile app: Share Sheet integration for Instagram/X and other applications.
- Conversational assistant: natural voice/text queries over the user's entire saved-resource library.
- Deeper integrations: official platform APIs where available and permitted.
- Resource intelligence: deduplication, related-resource recommendations, automatic categorization and

improved ranking.

15. PROPOSED TECHNOLOGY STACK

- Frontend: React, TypeScript, Vite, Tailwind CSS.
- Backend: Django + Django REST Framework.
- Database: PostgreSQL.
- Storage: Amazon S3.
- AI: LLM for summarization, metadata extraction, tagging and query understanding; embeddings for semantic retrieval.
- Speech: Gnani STT for Reel transcription and voice queries; TTS for conversational responses.
- Infrastructure: AWS, Docker, Redis/background task processing where required.

16. MVP ACCEPTANCE CRITERIA

- User can create an account and access a private resource library.
- User can add an Instagram Reel/Post or X/Twitter Post as a source.
- User can optionally provide creator information.
- User can attach an external resource URL.
- User can upload a supported document.
- Uploaded documents are stored in S3 and can be retrieved securely.
- System processes supported source/document content asynchronously.
- System generates and stores AI descriptions and relevant metadata.
- Reel audio can be transcribed when source/audio access is available.
- User can search saved resources using metadata and content.
- User can open the original source and retrieve the saved resource.
- Architecture supports future semantic and voice retrieval without major redesign.

17. PRODUCT SUCCESS DEFINITION

The primary MVP success criterion is not simply how many resources users save. It is whether users can successfully retrieve a resource later based on what they remember about it.

For example, a user should be able to remember “that resume template I saw on Instagram a few weeks ago” and have FindThatShit surface the relevant resource without requiring the exact filename, creator handle, or original URL.

The long-term product vision is:

Discover → Save → Understand → Organize → Remember → Retrieve

FindThatShit ultimately becomes a personal memory layer for useful resources discovered across the internet.